

UNIVERSITY OF GUADALAJARA

University Center for Exact Sciences
and Engineering

Division of Basic Sciences
Bachelor's Degree in Mathematics



Integrative Project in Mathematical Modeling

Comparative Analysis of Shallow versus Deep Neural Networks Using the Perceptron for Classification Problems

Cristobal Cortés Gómez

Advisor: Dr. Fernando Ignacio Becerra López

Guadalajara, Jal., December 2025

1 Justification

When choosing a network architecture to solve a classification problem, two of the most important parameters are the number of neurons and the number of hidden layers the neural network will have. For this purpose, simple heuristics are commonly used, such as the idea that more layers lead to better generalization Mhaskar, Liao, and Poggio 2017, while fewer layers make the network more manageable in terms of interpretability of what each neuron does, more stable under small changes in weights, and computationally efficient.

In general, it is known that so-called *shallow* networks — that is, those with a small number of hidden layers, or even just one — are already universal approximators Cybenko 1989, using as many neurons in those layers as needed. It also turns out that deep networks — using a limited number of neurons per layer but increasing the number of layers as much as needed — are likewise universal approximators Lu et al. 2017. Universality implies the ability to solve any classification problem Cybenko 1989, so in fact both architectures (shallow and deep) are viable options when solving a classification problem.

Analyzing what happens inside a neural network is a topic of broad study and high difficulty, since the networks themselves are often described as black boxes Nielsen 2015. For this reason, although the universal theorems are old, the topic of convergence continues to be studied, so that in recent years new observations of solution behavior keep appearing, such as the so-called *grokking* Fan 2024, which contradicts the established idea that one should not overfit training data, or that overfitting is generally a bad sign. Studies show that deep networks are more susceptible to this phenomenon and, additionally, others cite that they can exhibit benign overfitting Chatterji and Long 2023. For reasons such as these, the study of how neural networks function remains highly relevant and of great interest.

In addition to the above, one must keep in mind that other parameters can influence network behavior and lead us to different conclusions for the same architecture, since the cost function or even the number of sample points can affect which architecture is better, shallow or deep Yang and He 2024. Likewise, it is understood that the choice of activation functions broadly changes solution behavior depending on whether the network is shallow or deep Radhakrishnan, Belkin, and Uhler 2023.

It should be noted that the discussion of which architecture is better has in recent years pointed toward deep networks with the so-called *deep learning* Kůrková 2020; however, the debate continues, since it has been shown that various deep networks can be implemented in shallow versions Ba and Caruana 2014, which means the question of which architecture to choose remains open.

2 Objectives:

2.0 Main Objective

To investigate the vast range of possibilities between shallow and deep networks, focusing on the resolution of classification problems using a sigmoidal perceptron, in order to understand the differences between the two approaches.

2.1 Specific Objectives

- Find a fair metric for comparing shallow and deep networks.
- Apply the comparison to various datasets that clearly reveal the differences between solutions, verifying what the literature says about the better generalization of deep networks Mhaskar, Liao, and Poggio 2017. Contrast the advantages of using a shallow network against real-world scenarios.
- Implement the methodology proposed in Ba and Caruana 2014 to transform deep networks into shallow ones and verify the computational efficiency.
- Study the computational efficiency of both types of networks, considering, where possible, how advantageous one is over the other on current hardware.

3 Hypothesis

To understand both approaches well enough to distinguish which is the ideal architecture for solving a real classification problem. That is, when given a specific classification problem, to be capable of *a priori* selecting the appropriate architecture type through a well-described methodology or procedure, taking computational cost into account at all times.

4 Methodology

- Build a program that trains a perceptron on a specific dataset to solve a classification problem, where the number of layers and neurons per layer can be varied easily.
- Find a classification problem with the necessary complexity and a sufficient but not excessive amount of data for conducting tests.
- Develop a measurement methodology that allows us to decide which networks best solve our classification problem.

- Use the created program, the classification problem, and the metric with network architectures of varying numbers of layers and neurons, in order to produce statistics showing how solutions improve or worsen as neurons or layers are added or removed.
- Test the transformation of a deep network into a shallow one and, if possible, vice versa.
- Test with other classification problems whether the conclusions from the first one hold or whether there is no consistency in the methodology.
- Study the computational cost of the architectures that best solve the problem against others.
- Conclude with a recipe or, if none exists, conclude what other methods can be explored to address the problem of deciding the best architecture between a shallow or deep network for a specific classification problem.

5 Schedule:

N	Activity	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov
1	Review state of the art											
2	Build program											
3	Select problems											
4	Find metric											
5	Run experiment											
6	Verify costs											
7	Write document											

6 Development

A multilayer perceptron, or MLP (*Multi-Layer Perceptron*), is the name given to the fundamental artificial neural network from which all modern artificial networks are built, from those that process images to those that process natural language, as seen with the popular ChatGPT. These networks have an incredible range of applications, proving useful in virtually every corner of knowledge and everyday life.

This network is composed of artificial neurons modeled by **perceptrons**, such that for a given input, the neurons process it and return a corresponding output. The neurons connect to one another forming a network through which information is transmitted from input to output. The network is unidirectional (*feedforward*) and the neurons are organized into layers: inputs connect to the neurons of the first layer, which in turn connect to the neurons of the second, and so on until the neurons of the last layer produce the model's outputs. The intermediate neuron layers are called *hidden layers*, while the last layer is called the output layer.

6.1 The Perceptron and the Mathematical Model

The **perceptron** is the most basic model of an **artificial neuron** and a fundamental component in the field of artificial intelligence and machine learning. It was proposed by psychologist and computer scientist Frank Rosenblatt in the late 1950s. A perceptron is a computational model designed to mimic the basic functioning of a biological neuron; it receives multiple numerical inputs, processes them, and produces a single output. It is the fundamental unit from which more complex artificial neural networks are built.

The functioning of the perceptron is based on several key components:

- **Inputs (x_i):** Numerical values provided to the model, coming from the data or from previous layers.
- **Weights (w_i):** Each input has an associated synaptic weight that determines its relative importance in the neuron's activation. During training, these weights are adjusted to minimize the output error.
- **Weighted sum:** The perceptron computes the linear combination of the inputs multiplied by their respective weights.
- **Bias (b):** An additional parameter that allows shifting the activation function, analogous to the firing threshold in a biological neuron.
- **Activation function (σ):** The result of the weighted sum is passed through a nonlinear activation function that determines the intensity of the neuron's output.

Mathematically, the response of a perceptron is given by:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right)$$

where x_i represents each component of the input vector, w_i the synaptic weight associated with that input, and b the bias. The function σ represents the nonlinearity of the system. It is important to note that the activation function σ is applied only after the linear transformation (weighted sum), and is usually the same for all neurons in a given layer, although it may vary between different layers.

Regarding the MLP neural network, its modeling follows from connecting the response of each perceptron in one layer to the input of each perceptron in the next, until the entire network is complete, considering that the first perceptrons are connected to the direct inputs of the model. The simplest example of a multilayer network would have a single perceptron/neuron per layer. Its response, for a network of n layers, would be given by the composition of functions:

$$y = \sigma^{(n)} \left(w^{(n)} \dots \sigma^{(2)} \left(w^{(2)} \sigma^{(1)} \left(w^{(1)} x + b^{(1)} \right) + b^{(2)} \right) \dots + b^{(n)} \right)$$

To generalize this to multiple neurons per layer, we use matrix notation. Defining $\mathbf{W}^{(l)}$ as the weight matrix of layer l , $\mathbf{b}^{(l)}$ as the bias vector, and $\mathbf{a}^{(l)}$ as the activation vector (outputs) of layer

l , the forward propagation is defined recursively as:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

where $\mathbf{a}^{(0)} = \mathbf{X}$ (input). Thus, the network consists of multiple affine transformations followed by pointwise nonlinearities. In a sense, the MLP extends Linear Regression models and, more specifically, the Logistic Regression model, enhancing them through the capacity to represent complex nonlinear functions via composition.

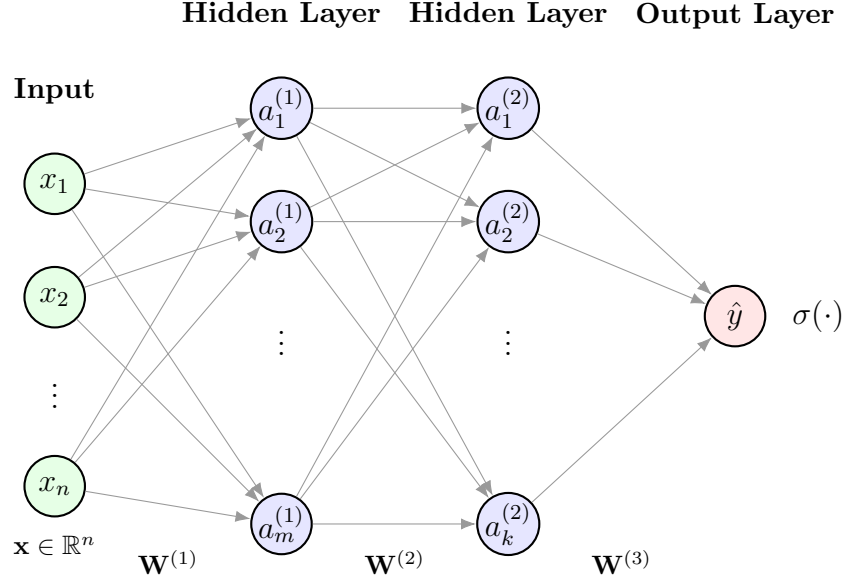


Figure 1: Schematic diagram of a deep Multilayer Perceptron (MLP). Information flows from the input layer (left) through successive hidden layers to the output layer (right). Each connection represents a synaptic weight w_{ji} .

To simplify the analysis and notation in optimization problems, it is common to group all adjustable parameters of the model (weights and biases from all layers) into a single parameter vector denoted θ . In this way, we can refer to the complete neural network as a parameterized function $f(x; \theta)$.

The model of our network is ultimately a *function* $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$. The behavior of this function is completely determined by the values in θ . Moreover, given the nature of linear transformations together with the fact that the functions σ (such as the sigmoid) are continuous, the MLP model is, in its entirety, a continuous function.

6.2 Universality and Functional Analysis

In terms of mathematical analysis, there are many questions we can ask about the function represented by the network. The most important is whether a complete correspondence exists: we already know that the function we obtain is continuous, but is it possible for any continuous function we wish to approximate to be representable by a neural network?

This question motivated many studies in the second half of the 20th century. It was formally proved by Cybenko in 1989, who used functional analysis for this purpose and emphasized that this area of mathematics could be used to address the theory of neural networks. The proof extends this result to functions beyond continuous ones under certain measurability conditions.

Formally, Cybenko’s Universal Approximation Theorem states that given a neural network with a single *hidden layer* of neurons represented by perceptrons with *sigmoidal* activation functions, it can approximate any continuous function f on a compact subset of $\mathbb{R}^n$. This means that single-hidden-layer neural networks are dense in the space of continuous functions $C(K)$. The fundamental inequality is expressed as:

$$|g(x) - f(x)| < \epsilon \quad \forall x \in K$$

where $g(x)$ represents the output function of the neural network (the finite sum of the form $\sum \alpha_j \sigma(y_j^T x + \theta_j)$) and ϵ is an arbitrarily small error. This guarantees the existence of a network that approximates the target function, although it does not tell us how to find the weights.

The proof continues with results showing density in spaces of integrable functions (L^2). These theorems are popularly known as Universal Approximation Theorems, and a corollary of them is that any *classification problem* can be solved. That is, given a dataset where each example belongs to a class, it is possible to find a function that maps each data point one-to-one to its respective class (e.g., classifying images of animals).

It is worth noting that, in the context of statistical learning, it is conventional to denote the complete set of adjustable parameters of the model (weights w and biases b) by the vector θ . This notation highlights the interpretation of the model as a probabilistic approximation, $P(y|x;\theta)$, especially useful in scenarios where the underlying data-generating function is unknown and one works with samples that contain inherent noise. In this work we will adopt that notation to refer to the model parameters when we do not need to explicitly mention the weights W and biases b .

For this thesis we will focus on *binary classification problems*, where the dataset is associated with only two classes: 0 and 1 (light and dark colors, positive and negative diagnoses, etc.).

Historically, the fact that networks with one or two layers had theoretical results demonstrating such strong capabilities temporarily caused interest in exploring networks with more layers to wane, under the premise of “why add more if what we have is sufficient?”. However, the rise of better processing technologies — specifically GPUs — revived interest in these networks by showing them to be empirically superior for solving problems with very large datasets, giving birth to *Deep Learning*. Later, complementary theorems Lu et al. 2017 appeared that demonstrated universal approximation capacity from the depth perspective (bounded width, variable depth).

Many modern theorems such as the one mentioned Lu et al. 2017 do not use sigmoidal activation functions, relying instead on functions like ReLU (*Rectified Linear Unit*), which describes a ramp function ($f(x) = \max(0, x)$). Nevertheless, the function used in this document is the logistic (sigmoid) function, which is differentiable everywhere on its domain, with range $(0, 1)$, strictly increasing, and antisymmetric with respect to its midpoint, modeling the classical neural activation probability where neurons respond by activating at 1 and deactivating at 0.

6.3 The Optimization Problem and Learning

The motivation of this thesis lies in the competition between shallow networks (which approximate by adding neurons to a fixed layer) and deep networks (which approximate by adding layers of fixed width). While theory guarantees the *existence* of a network for both cases, in practice we face the problem of finding the actual values of the weights θ .

Here a crucial distinction arises: Cybenko’s proof is existential, not constructive. It tells us that a configuration of weights that solves the problem “exists”, but finding it is an optimization challenge. For this, we have algorithms that seek to minimize a **cost function** (or loss function), which measures the discrepancy between the network’s prediction and the expected true value.

The standard method for finding these weights is *gradient descent*. Since our network is a differentiable function (thanks to the use of sigmoids), we can compute the gradient of the cost function with respect to all parameters, $\nabla_{\theta}J(\theta)$. This vector indicates the direction of maximum error growth, so we update the weights in the opposite direction:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta}J(\theta)$$

where η is the learning rate.

However, parallel problems arise when trying to find solutions. One is overfitting (*overfitting*), where the network learns the noise of the training data instead of the underlying function. Another is the difficulty of the optimization landscape: even if a network has the theoretical capacity to represent the function, the gradient descent algorithm may get trapped in suboptimal local minima or regions of zero gradient, preventing the ideal parameters from being found.

During the literature review, theoretical formulas have been identified that allow bounding the minimum number of neurons needed for a given architecture to be capable of approximating the target function. Nevertheless, practically obtaining the solution faces two fundamental barriers that must be clearly distinguished in our analysis.

The **first problem is statistical and concerns generalization**. Since in real situations we only have a finite set of examples, often contaminated by noise, the learning algorithm tends to find a set of empirical parameters $\hat{\theta}$ that minimizes the error over the available sample (interpolation), but which does not necessarily correspond to the ideal parameters θ^* that minimize the error over the true data distribution (extrapolation). That is, there is a risk that the network learns to describe the particularities and noise of the data subset instead of capturing the underlying generating function.

The **second problem concerns optimization and reachability**. Even assuming that the gap between $\hat{\theta}$ and θ^* were negligible, there is no guarantee that current optimization methods, such as gradient descent, are capable of navigating the cost function landscape to find that global solution. This is notably evidenced in the literature on *model compression* Ba and Caruana 2014: it has been shown that it is possible to “distill” the knowledge of a complex deep network into a shallow network, which mathematically proves that the shallow network possesses the *theoretical capacity* to represent the solution. However, it turns out to be extremely difficult, if not impossible, to train that same shallow network from scratch to reach that level of performance. This suggests that the limiting factor lies not in the representational capacity of the architecture, but in the

inefficiency of the search algorithm in finding minima in shallow architectures without external guidance.

In conclusion, we consider it essential to address both challenges simultaneously: it is not sufficient to verify that a network has the theoretical capacity to solve the problem; it is also necessary to validate whether it is feasible to reach that solution with the available optimization algorithms. Unlike previous studies that usually isolate these variables, this work seeks to contrast both limitations to determine the most appropriate architecture for solving the classification problem.

6.4 The Binary Classification Problem

When we speak of a binary classification problem, we refer to one with specific conditions. In this type of problem, a training dataset is available in which each example consists of a pair $(\mathbf{x}, y)$, with $\mathbf{x} \in \mathbb{R}^n$ corresponding to the input to be evaluated by the network, and $y \in \mathbb{R}$ (typically $y \in \{0, 1\}$) corresponding to the expected output or target class.

These components for the entire dataset can be represented by an input matrix $\mathbf{X}$ and an output vector $\mathbf{Y}$. If we have k example data points, we organize the matrix $\mathbf{X}$ with dimensions $(n \times k)$ by arranging each input vector $\mathbf{x}$ as a column, while the responses form a row vector $\mathbf{Y}$ of dimension $(1 \times k)$, maintaining the column order in $\mathbf{X}$.

This implies that if f is the ideal function that solves the classification problem such that $f(\mathbf{x}) = y$, then for the complete dataset:

$$F(\mathbf{X}) = \mathbf{Y}$$

where $F(\mathbf{X}) = (f(\mathbf{x}_1), f(\mathbf{x}_2), \dots, f(\mathbf{x}_k))$.

Thus, the optimization problem we want to solve when training the neural network is to find the parameterized function G_θ that best approximates f , minimizing a cost function. Using the Mean Squared Error, we seek:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} (||G_\theta(\mathbf{X}) - \mathbf{Y}||^2)$$

Here we assume that G_θ is the construction of our neural network that attempts to approximate f , so that after training $G_\theta(\mathbf{x}) \approx y$.

It is essential to emphasize that the fact that the network correctly solves this limited set of examples does not imply that the function will extrapolate correctly to examples the network has not been trained on. Moreover, due to the natural noise present in data collection, attempting to reduce the cost function to an absolute zero is generally counterproductive, as the network would learn to predict the inherent noise, worsening its generalization ability.

With this in mind, the process for finding the weights and biases that best approximate the function f is carried out using the **gradient descent** algorithm. The iterative update rule is:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta ||G_\theta(\mathbf{X}) - \mathbf{Y}||^2$$

where η is the learning rate and θ_0 is arbitrary, typically random. The parameter vector θ is expressed explicitly as the concatenation of all the weights $\mathbf{w}$ and biases $\mathbf{b}$ of the network:

$$\begin{aligned}\theta &= [\mathbf{w}, \mathbf{b}] \\ &= (w_{11}^{(1)}, \dots, w_{nm}^{(L)}, b_1^{(1)}, \dots, b_m^{(L)})\end{aligned}$$

where L is the total number of layers (including the output layer). The network function $G_\theta(\mathbf{X})$ is defined recursively as a composition of linear transformations and activations:

$$G_\theta(\mathbf{X}) = \sigma^{(L)} (\mathbf{W}^{(L)} \dots \sigma^{(1)} (\mathbf{W}^{(1)} \mathbf{X} + \mathbf{B}^{(1)}) \dots + \mathbf{B}^{(L)})$$

The gradient ∇_θ needed for the update is obtained via the chain rule (the *Backpropagation* algorithm), yielding a vector containing the partial derivatives of the cost function with respect to each component of θ :

$$\nabla_\theta J = \left(\frac{\partial J}{\partial w_{11}^{(1)}}, \dots, \frac{\partial J}{\partial w_{nm}^{(L)}}, \frac{\partial J}{\partial b_1^{(1)}}, \dots, \frac{\partial J}{\partial b_m^{(L)}} \right)$$

Once the mathematical mechanism for adjusting parameters and minimizing the theoretical error over known data has been defined, it becomes necessary to interpret the network’s output to make decisions and evaluate how well the model generalizes to new data.

Finally, to operationalize the evaluation of architectures in the context of binary classification, it is necessary to translate the network’s probabilistic output into a concrete categorical decision. Since our output function is sigmoidal, the network delivers a continuous value $\hat{y} \in (0, 1)$ representing the conditional probability $P(y = 1|\mathbf{x}; \theta)$. To assign a definitive class label $y_{pred} \in \{0, 1\}$, we establish a decision threshold (typically $\tau = 0.5$), defined by the rule:

$$y_{pred} = \begin{cases} 1 & \text{if } \hat{y} \geq \tau \\ 0 & \text{if } \hat{y} < \tau \end{cases}$$

The validation of the architecture is not performed uniformly over the entire available dataset. To measure generalization capacity — which is the central criterion for comparing deep versus shallow networks — the dataset is partitioned into two disjoint subsets: a *training* set, used exclusively for the iterative adjustment of parameters θ via gradient descent, and a *test* (or validation) set, which remains “invisible” to the network during the learning phase and is used only for the final evaluation.

The critical analysis is based on contrasting the performance metrics obtained on these two sets. It is in the difference or “gap” between both results where the structural behavior of the model is diagnosed:

- High performance on the training set accompanied by degraded performance on the test set indicates **overfitting**. This implies that the network has “memorized” the noise or specific particularities of the training data, losing the ability to extrapolate to new examples.

- Poor performance on both sets suggests **underfitting**, which indicates that the architecture lacks the representational capacity needed to model the complexity of the problem, or that the optimization algorithm has failed to converge.

Therefore, the optimal architecture is not necessarily the one that minimizes training error to zero, but rather the one that achieves the lowest possible test error while maintaining a minimal generalization gap. Since simple accuracy can be misleading, especially in datasets where one class is dominant, in the following sections we will define more robust metrics (Precision, Recall, and F1-Score) that will serve as the quantitative standard for our comparison methodology.

6.5 Computational Component

To put the gradient formulas into practice and obtain the vector $\nabla_{\theta} J$ efficiently, we use the **Back-propagation** algorithm. This method computes the partial derivatives of the cost function by applying the chain rule recursively, starting from the output layer and working backward to the first layer.

The algorithm is fundamentally based on computing the error term $\delta^{(l)}$ for layer l using the already-computed error from the next layer ($l + 1$):

$$\delta^{(l)} = ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$$

where $\odot$ represents the element-wise multiplication (Hadamard product) and $\mathbf{z}^{(l)}$ is the weighted input to the neuron. With these errors, the final derivatives for the weights are obtained via matrix products.

Analyzing these operations reveals that the computational effort is considerable and depends directly on the problem size. In terms of asymptotic complexity, the cost of performing a complete training step called an epoch — that is, computing θ_{i+1} — scales as $O(k \cdot w)$, where k is the number of training examples and w is the total number of parameters (weights and biases) in the network Goodfellow, Bengio, and Courville 2016.

For this reason, we decided to include computational cost as a key point in our comparison. We are interested in contrasting that, even though two networks may have the same total number of parameters w (and hence the same theoretical complexity), their execution on real hardware differs: **shallow** networks perform few operations with very large matrices — which is highly parallelizable on GPUs — while **deep** networks involve a long sequence of operations with smaller matrices, where each layer must wait for the previous one, introducing sequential latency.

7 Methods for Conducting the Analysis

7.1 Network Programming

7.1.1 A Versatile Program as the Experimental Base

To carry out this study following the stipulated methodology, a versatile software tool was implemented that allows neural networks to be created, stored in memory, evaluated, and optimized via gradient descent using the *Backpropagation* algorithm.

This program was designed with flexibility in mind: all network *hyperparameters* are easily configurable. It also implements additional tools to enhance gradient descent performance, such as stochastic variants (SGD), adaptive learning rates, customizable stopping conditions, and the ability to swap cost and activation functions. It even includes a numerical gradient verification module to ensure that the complex analytical computation of derivatives is being executed correctly.

The main input of the program is the definition of the network architecture, which is specified via an ordered tuple or list of integers. This list represents the number of neurons in each layer, explicitly including the input and output layers. For example, the tuple (5, 3, 10, 1) defines a network with 5 inputs, a first hidden layer of 3 neurons, a second hidden layer of 10 neurons, and a single output.

Once the architecture is defined, training is performed by supplying the corresponding data as pairs of matrices $\mathbf{X}$ and $\mathbf{Y}$ composed of floating-point numbers.

7.1.2 A Second, More Powerful but Limited Program

Because the selected classification problems have, on average, a few thousand examples, and because the experiments ended up running for up to one million epochs, a second program was developed that processes data in ultra-parallel using the GPU to save time during testing.

This second program was designed exclusively for computational efficiency and is more limited, but has everything necessary to execute gradient descent and compute the F1-Score.

7.2 Selection of Classification Problems

It was determined that the classification problems to be used should not require complex preprocessing; the data should be ready to be fed into the network directly upon download. This decision arose from an initial experience with the Titanic survivors dataset, which required extensive treatment that introduced uncertainty about whether failures in solving the problem were due to the network architecture or the quality of the preprocessing.

Subsequently, it was concluded that to carry out a better comparison between different archi-

tructures, it was necessary to measure performance in a variety of contexts. Therefore, an initial collection of 7 classification problems was selected, to which 7 more of medium difficulty were later added, forming a diverse test bank.

7.3 Comparison Methodology

When designing the comparison, we were faced with the enormous number of possible combinations of neurons and layers. Since testing all of them is not feasible, the first challenge was to define which specific networks to contrast in order to ensure a fair analysis. It was established that the comparison should be 1-to-1 between shallow and deep networks, normalizing **computational cost** as the equivalence criterion.

Our fundamental premise was that the most reliable indicator of computational cost is the total number of weights (parameters) in the network, since this number is directly proportional to the number of multiplication and addition operations required to evaluate the model.

To simplify the analysis, we imposed a structural restriction: deep networks would have a constant number of neurons per hidden layer, while shallow networks would consist of a single hidden layer with a variable number of neurons.

The problem was formulated mathematically as follows:

Consider a pair of networks with input dimension N and output dimension M .

- A **shallow** network with a single hidden layer of L neurons.
- A **deep** network with K hidden layers, each with $\hat{L}$ neurons.

The total number of weights $\#w$ for each architecture is given by:

$$\#w = \begin{cases} NL + LM & \text{For the shallow network} \\ N\hat{L} + (K - 1)\hat{L}^2 + \hat{L}M & \text{For the deep network} \end{cases}$$

Our goal is to equalize the number of parameters of both architectures to make them computationally equivalent. Equating the expressions and solving for L , we obtain the necessary relation for the shallow network:

$$L = \frac{N\hat{L} + (K - 1)\hat{L}^2 + \hat{L}M}{N + M}$$

As an example, if $N = 5$, $M = 1$, $\hat{L} = 3$, and $K = 3$:

$$L = \frac{15 + 18 + 3}{5 + 1} = \frac{36}{6} = 6$$

This means that a deep network with 3 hidden layers of 3 neurons each is equivalent in number of parameters to a shallow network with a single hidden layer of 6 neurons.

Now, a question arises: which architecture uses more neurons in total? The total number of neurons in the deep network is $K\hat{L}$, while in the shallow network it is simply L .

To find out, we state the inequality assuming the deep network has more or equal neurons ($\hat{L}K \geq L$) and examine under what conditions it holds:

$$\begin{aligned}\hat{L}K &\geq L \\ \hat{L}K &\geq \frac{N\hat{L} + (K-1)\hat{L}^2 + \hat{L}M}{N+M}\end{aligned}$$

Dividing the entire expression by $\hat{L}$ (which is positive):

$$\begin{aligned}K &\geq \frac{N + K\hat{L} - \hat{L} + M}{N+M} \\ K(N+M) &\geq N + M - \hat{L} + K\hat{L} \\ K(N+M) - K\hat{L} &\geq N + M - \hat{L} \\ K(N+M - \hat{L}) &\geq N + M - \hat{L}\end{aligned}$$

At this point, to solve for K we must divide by the term $(N + M - \hat{L})$. Since dividing an inequality by a negative number reverses the sign, we have two possible outcomes for K :

$$K \begin{cases} \geq 1 & \text{if } \hat{L} < N + M \quad (\text{the term is positive}) \\ \leq 1 & \text{if } \hat{L} > N + M \quad (\text{the term is negative}) \end{cases}$$

Therefore, whether the deep network will have more total neurons than the shallow network depends exclusively on whether the number of neurons per hidden layer $\hat{L}$ is less than or greater than the sum of the input and output dimensions $N + M$.

- If it is smaller ($\hat{L} < N + M$), the condition always holds, since by definition $K \geq 1$. In this case, the deep network has **more** neurons.
- If it is larger ($\hat{L} > N + M$), the condition $K \leq 1$ would only hold if $K = 1$. For any truly deep network ($K > 1$), the inequality does not hold, which means the shallow network has **more** neurons.

Concretely, for networks with the same number of parameters:

$$\begin{array}{ll} \text{Deep Neurons} > \text{Shallow Neurons} & \text{if } \hat{L} < N + M \\ \text{Deep Neurons} < \text{Shallow Neurons} & \text{if } \hat{L} > N + M \\ \text{Deep Neurons} = \text{Shallow Neurons} & \text{if } \hat{L} = N + M \end{array}$$

We verify this last equality by substituting into the original equation for L :

$$L = \frac{N\hat{L} + (K-1)\hat{L}^2 + \hat{L}M}{N+M} = \frac{\hat{L}(N+M) + (K-1)\hat{L}^2}{N+M}$$

If $\hat{L} = N + M$, then:

$$L = \frac{\hat{L}^2 + (K-1)\hat{L}^2}{\hat{L}} = \hat{L} + (K-1)\hat{L} = K\hat{L}$$

which confirms that if $\hat{L} = N + M$, both networks have exactly the same number of neurons and weights.

This rule provides a clear strategy for experimental design. For this project, we decided to **compare networks that have both the same number of weights and the same number of neurons**. This is achieved by defining the deep architecture so that the width of its hidden layers equals the sum of the input and output dimensions: $\hat{L} = N + M$. Since we work with binary classification problems, the output is always $M = 1$, so we fix $\hat{L} = N + 1$.

In summary, the experiment consists of taking a problem with input dimension N , building deep networks with constant width of $N + 1$ neurons, and comparing them against equivalent shallow networks that have $L = K(N + 1)$ neurons in their single hidden layer.

7.3.1 One-to-One Comparison: Shallow vs. Deep

To compare the results of two architectures, we first train both networks. Since training is a stochastic process (depending on random initialization and data ordering), it must be repeated multiple times to obtain statistically significant results. At least 5 training runs are performed for each architecture on the given problem.

The primary metric chosen is the *F1-Score*. This metric is more robust than simple accuracy, as it penalizes imbalances between precision (correctly predicted positives) and recall (actual positives detected). Its range is $[0, 1]$, where 1 indicates perfect performance.

The evaluation criteria are as follows:

- **Absolute Performance:** The *F1-Score* is observed on both the training set and the test set. We seek values close to 1 in both cases.
- **Generalization Gap:** The difference between the training score and the test score is analyzed. A large difference is a clear indicator of *overfitting*, which is considered a negative result.
- **Stability and Convergence:** The number of iterations (epochs) needed to reach convergence is taken into account. The variance in this parameter across different runs speaks to the robustness of the architecture in consistently solving the problem.

It is worth noting that the evaluation on these three aspects may be independent; it could be the case that one architecture is preferable for its stability or generalization capacity, even if its absolute score is slightly lower.

7.3.2 Overall Rating

Since for each classification problem different configurations will be tested (varying depth K and the corresponding width L), we will analyze the progress of performance as network complexity increases. The theoretically expected outcome is that, upon reaching a certain number of neurons, performance stabilizes, indicating that the network has reached the *representational capacity* needed to approximate the underlying function of the problem.

8 Theoretical Framework

Below, the formal definitions and mathematical notation underlying the project's development are presented.

Definition 1 (MLP Neural Network Model): A Multilayer Perceptron (MLP) is a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, composed of a sequence of L affine transformations followed by pointwise nonlinear activations. For an input $\mathbf{x} \in \mathbb{R}^n$, the network is defined recursively as:

$$\begin{aligned}\mathbf{a}^{(0)} &= \mathbf{x} \\ \mathbf{z}^{(l)} &= \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \\ \mathbf{a}^{(l)} &= \sigma(\mathbf{z}^{(l)}) \quad \text{for } l = 1, \dots, L\end{aligned}$$

where $\mathbf{W}^{(l)}$ is the weight matrix, $\mathbf{b}^{(l)}$ the bias vector, and σ the activation function. The output of the network is $\mathbf{a}^{(L)}$.

Definition 2 (Logistic Sigmoid Function): The logistic function $\sigma : \mathbb{R} \rightarrow (0, 1)$ is defined by:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This function is monotonically increasing, bounded, and differentiable, with derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Definition 3 (Gradient): Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a differentiable function. The gradient of f at $\mathbf{x}$, denoted $\nabla f(\mathbf{x})$, is the vector field containing the partial derivatives of f :

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right]$$

Definition 4 (Local and Global Minima): Given a cost functional $J(\theta)$, a point θ^* is a global minimum if $J(\theta^*) \leq J(\theta)$ for all θ in the domain. It is a local minimum if there exists a neighborhood V around θ^* such that $J(\theta^*) \leq J(\theta)$ for all $\theta \in V$. Training neural networks seeks to approximate the global minimum, although it often converges to local minima.

Definition 5 We define θ as the vector that concatenates all adjustable parameters (weights and biases) of the L layers of the network:

$$\theta = \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)}\} \in \mathbb{R}^d$$

where d represents the total dimension of the parameter space. Under a probabilistic interpretation, the neural network does not determine an absolute deterministic output, but rather models a conditional probability distribution $P(y|\mathbf{x}; \theta)$.

Definition 6 (Gradient Descent): It is a first-order iterative optimization algorithm for finding a local minimum of a differentiable function. The update rule is:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

where $\eta > 0$ is the learning rate.

Definition 7 (Quadratic Cost Function): For a regression or classification problem (where the output is treated as a continuous probability), the Mean Squared Error (MSE) is defined as:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)}(\theta)\|^2$$

where $\mathbf{y}^{(i)}$ is the true label and $\hat{\mathbf{y}}^{(i)}$ the network's prediction for example i .

Gradient Computation (Backpropagation): The gradient of the cost function with respect to the network weights is computed via the chain rule, propagating the error from the output backward:

$$\frac{\partial J}{\partial w_{jk}^{(l)}} = \frac{\partial J}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}$$

where $\delta_j^{(l)}$ represents the error term at neuron j of layer l .

Definition 8 (Classification Metrics): To evaluate binary classification performance, the following metrics based on the confusion matrix are used (True Positives TP , False Positives FP , True Negatives TN , False Negatives FN):

- **Precision:** Proportion of correctly predicted positive cases.

$$P = \frac{TP}{TP + FP}$$

- **Recall:** Proportion of actual positive cases correctly detected.

$$R = \frac{TP}{TP + FN}$$

- **F1-Score:** The harmonic mean of precision and recall, useful for imbalanced classes.

$$F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Definition 8 (Hadamard Product): It is a binary operation that takes two matrices of the same dimensions and produces another matrix where each element ij is the product of the ij elements of the original matrices. It is commonly denoted by the symbol $\odot$. If $\mathbf{A}$ and $\mathbf{B}$ are matrices of dimension $m \times n$, then:

$$(\mathbf{A} \odot \mathbf{B})_{ij} = A_{ij} \cdot B_{ij}$$

In neural networks, this operation is fundamental for applying the derivative of the activation function to each neuron in a layer simultaneously during backpropagation.

Definition 9 (Backpropagation Algorithm): Known as *Backpropagation*, it is the standard method for computing the gradient of the cost function $\nabla_{\theta} J$ in multilayer networks. It is based on the recursive application of the chain rule from the output layer toward the input, allowing the computation of how each individual weight affects the total error without needing to evaluate the entire network separately for each parameter.

Definition 10 (Big-O Notation and Complexity): The notation $O(g(n))$ is used to describe the limiting behavior of a function as its argument tends to infinity. In the context of algorithms, it describes an upper bound for execution time (or memory space) as a function of the input size. For example, saying that training is $O(k \cdot w)$ implies that the computation time grows proportionally to the product of the number of data points k and the number of weights w .

Definition 11 (Training Epoch): An epoch (*epoch*) is defined as a complete cycle in which the learning algorithm has processed the entire training dataset once. Since gradient descent is iterative, training a network usually requires multiple epochs for the cost function to converge to an acceptable minimum.

9 Instrumentation and Applied Strategies

9.1 Hardware Environment

To carry out the experimental tests we used two computing machines with different purposes. The first, dedicated to development and preliminary testing, was available 24/7. The second, a high-performance machine, was used for the massive resolution of training problems, taking advantage of its dedicated NVIDIA RTX 3090 Ti graphics card.

9.2 Software Implementation

Software development was divided into two main components to leverage the strengths of different languages:

1. Preprocessing and Prototyping (Python): The first program uses the *Pandas* library for reading and curating (cleaning) the datasets, subsequently transferring the information to the component responsible for executing the neural networks, which is written entirely using the *NumPy* library. This methodology proved extremely practical during the experimentation phase, as *Pandas* allows manipulating datasets and performing complex matrix operations with simple and efficient syntax.

2. High-Performance Training Engine (C++/CUDA): The second program was written directly in C/C++ using the CUDA architecture to exploit GPU parallelism. In an initial stage we attempted to implement ‘.csv’ file reading routines similar to those in the Python program; however, this approach turned out to be unnecessarily complex and tedious in C++. We ultimately opted for a hybrid strategy: the CUDA program receives datasets already preprocessed and standardized by a simpler auxiliary Python script.

9.3 File Management Strategy

This methodological decision led us to maintain a strict and standardized order in the directory and file structure where models and data are stored. This structural rigidity, far from being an inconvenience, greatly facilitated the collection of results and opened the possibility of automating preprocessing and post-processing with external tools.

To run the experiments, the compiled CUDA binary is copied to the root directory containing this file structure. Once there, the program sequentially launches multiple training processes, varying the number of neurons and layers according to the desired configuration until all requests are completed.

As each model’s training finishes, the system generates unique records identified by a *hash* code. These files store all relevant training information in a simple but readable format, ready to be interpreted by the analysis tools. This save architecture allows the process to be stopped at any time and resumed later without losing already-generated records.

Finally, the records are processed by a Python script that runs in the same folder, which is responsible for generating comparative plots and exporting spreadsheet files (Excel/CSV) with the raw data and their most relevant statistics.

9.4 Obtaining the Classification Problems

The binary classification problems used in this study were acquired using the *sklearn.datasets* library. This tool allowed us both to generate controlled synthetic datasets and to download standardized databases from public repositories.

STORE N GO New folder Tesis Problem6_Spiral Dividir		
Nombre	Tamaño	Modificado
Reg	54 elementos	02/12/25 a las 8:08 p.m.
New folder	0 elementos	02/12/25 a las 8:08 p.m.
Res	6 elementos	02/12/25 a las 7:04 p.m.
Bin	2 elementos	02/12/25 a las 4:27 p.m.
Train	2 elementos	02/12/25 a las 4:27 p.m.
Y	1 elemento	02/12/25 a las 4:27 p.m.
1x1600.bin	6.3 KiB	02/12/25 a las 9:21 a.m.
X	1 elemento	02/12/25 a las 4:27 p.m.
2x1600.bin	12.5 KiB	02/12/25 a las 9:21 a.m.
Test	2 elementos	02/12/25 a las 4:27 p.m.
Y	1 elemento	02/12/25 a las 4:27 p.m.
X	1 elemento	02/12/25 a las 4:27 p.m.
analisis_registros.py	9.8 KiB	02/12/25 a las 8:24 p.m.
Red.exp	712 B	02/12/25 a las 4:51 p.m.
Red.exe	384.0 KiB	02/12/25 a las 4:51 p.m.

Figure 2: Standardized file structure for problem 6 “Spiral”.

_2_6_1_1764697569.txt — ...		_2_3_3_1_1764691097.txt ...	
Archivo Editar Selección Ver Ir		Archivo Editar Selección Ver Ir	
Nuevo Abrir...		Nuevo Abrir...	
1		1	
2	Arquitectura: _2_6_1_	2	Arquitectura: _2_3_3_1_
3	Semilla: 1764697569	3	Semilla: 1764691097
4	Segundos: 108	4	Segundos: 351
5	Iteraciones: 500000	5	Iteraciones: 1000000
6		6	
7	Costo_train: 0.123278	7	Costo_train: 0.208237
8	Precision_train: 0.837532	8	Precision_train: 0.623377
9	Recall_train: 0.832290	9	Recall_train: 0.841051
10	F1_train: 0.834903	10	F1_train: 0.716036
11		11	
12	Costo_test: 0.123665	12	Costo_test: 0.209376
13	Precision_test: 0.815000	13	Precision_test: 0.617544
14	Recall_test: 0.810945	14	Recall_test: 0.875622
15	F1_test: 0.812968	15	F1_test: 0.724280
16		16	
17	Paso: 1.000000	17	Paso: 0.010000
18	Gamma: 0.300000	18	Gamma: 0.900000
19	Parada: 0.100000	19	Parada: 0.100000
20	Iteraciones: 500000	20	Iteraciones: 1000000
21	CapasDesde: 3	21	CapasDesde: 2
22	CapasHasta: 4	22	CapasHasta: 3
23	Repeticiones: 5	23	Repeticiones: 5
24	Intervalo: 10000	24	Intervalo: 10000
25	MultiplicadorProfunda: 0.700000	25	MultiplicadorProfunda: 0.700000
26	Tarjeta: NVIDIA GeForce GTX 950M	26	
27		27	Costos:
28	Costos:	28	0.3668880.3277400.3270870.3256200
19:17 INSERTAR Tabuladores débiles: 4 UTF-8 Normal		16:2 INSERTAR Tabuladores débiles: 4 UTF-8 Normal	

Figure 3: Comparison of record files: on the left a shallow network and on the right its deep network equivalent.

10 Results and Conclusions

The following presents the results obtained for each of the selected classification problems, comparing the performance of the shallow and deep architectures under the described experimental conditions.

10.1 Problem 1: Blobs

- **Type:** Synthetic (`make_blobs`).
- **Description:** Clearly separated point clouds.
- **Challenge:** Usually linearly separable. Used as a baseline control.

Results: All tested networks solved the problem without difficulty, achieving an *F1-Score* of 100%, validating the correct functioning of the training algorithm.

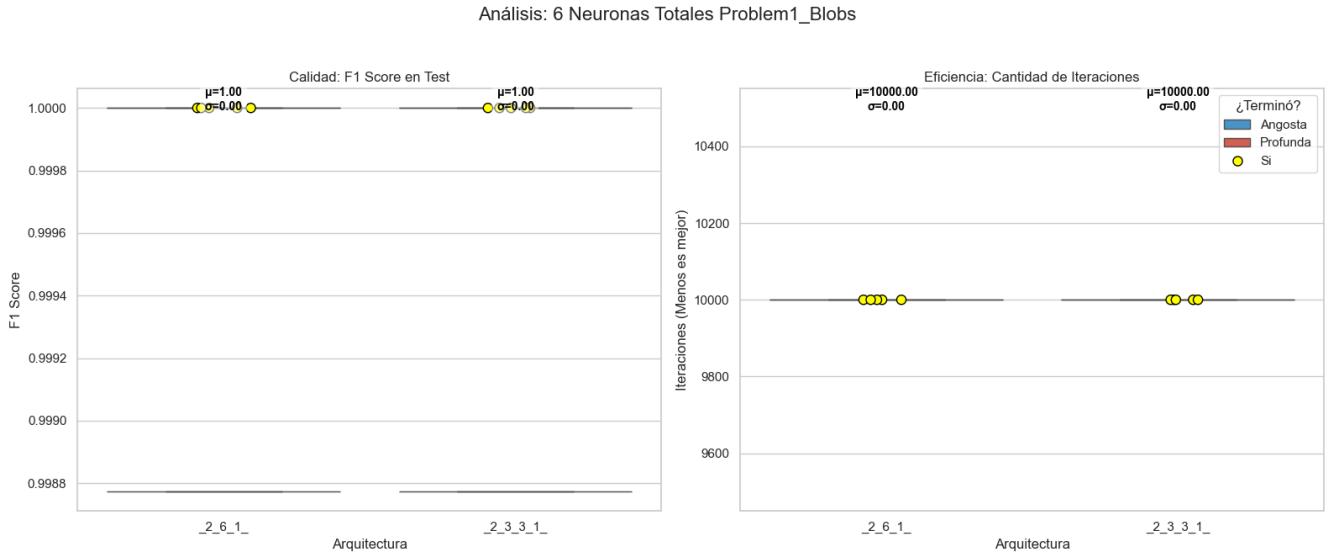


Figure 4: Performance comparison on problem 1 Blobs with 6-neuron networks.

10.2 Problem 2: Breast Cancer (Wisconsin)

- **Type:** Real dataset (Medical).
- **Description:** Features computed from a digitized image of a fine needle aspirate of a breast mass. Classifies between *Malignant* and *Benign*.
- **Challenge:** High dimensionality (30 numerical features). The challenge lies in avoiding overfitting and determining feature relevance.

Results: With 62 neurons, both architectures effectively solve the problem, achieving a mean close to 95%. Notably, test scores were slightly higher than training scores, indicating excellent generalization.

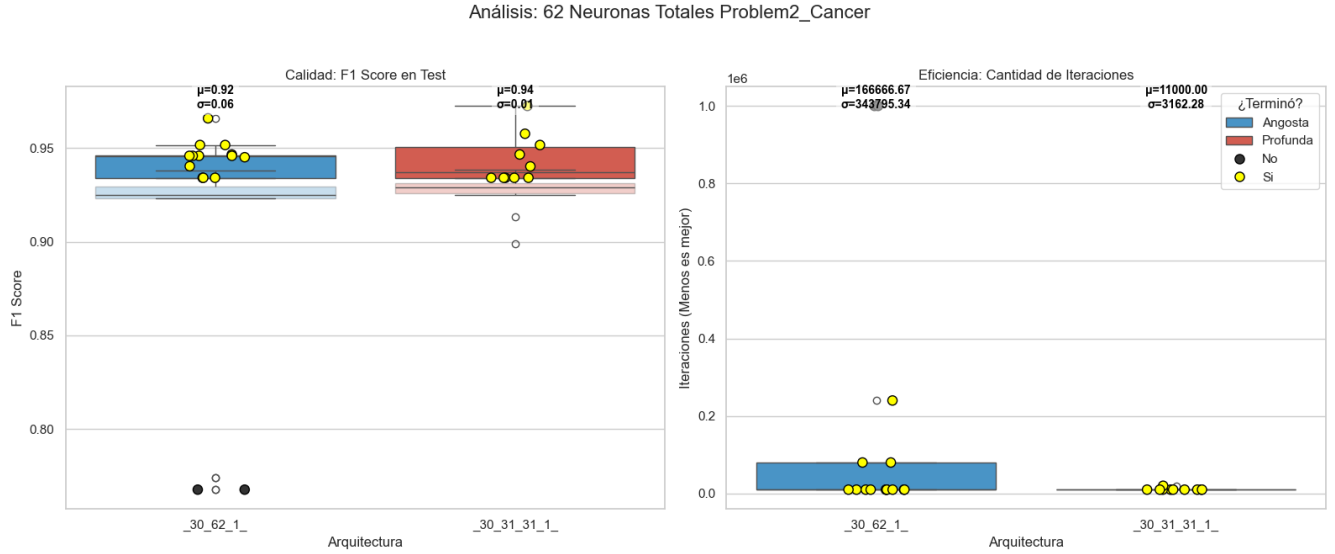


Figure 5: Performance comparison on problem 2 Breast Cancer with 62-neuron networks.

10.3 Problem 3: XOR (Exclusive Or)

- **Type:** Synthetic (Logical).
- **Description:** Four groups of points where diagonal corners belong to the same class.
- **Challenge:** Classic nonlinearity case study. Impossible to solve with a single straight line; hidden layers are strictly required.

Results: With only 6 neurons, both networks solve the problem with scores of 99.9%.

10.4 Problem 4: Circles

- **Type:** Synthetic (`make_circles`).
- **Description:** A circle inscribed within a concentric ring.
- **Challenge:** Requires a closed radial decision boundary.

Results: [Results pending in the original text].

10.5 Problem 5: Moons

- **Type:** Synthetic (`make_moons`).
- **Description:** Two interleaved semicircles.
- **Challenge:** Evaluates the ability to model curved boundaries and local nonlinearities.

Results: Both models solve the problem with a score of approximately 89%. No significant improvement is observed when adding more neurons, suggesting that the problem's complexity has been saturated.

10.6 Problem 6: Spiral

- **Type:** Synthetic.
- **Description:** Two or more spiral arms emanating from the center.
- **Challenge:** Considerably more difficult than the previous problems. Requires understanding connectivity along a continuous curved trajectory without jumping between adjacent arms.

Results: This is the first problem with sufficient difficulty to reveal structural differences between architectures.

With 6 neurons, a mean of 72 points is obtained for both architectures. However, the standard deviation of the shallow network is 8 points, while the deep network shows greater stability with only 4. There are no signs of overfitting and no network completed training before the 1-million epoch limit.

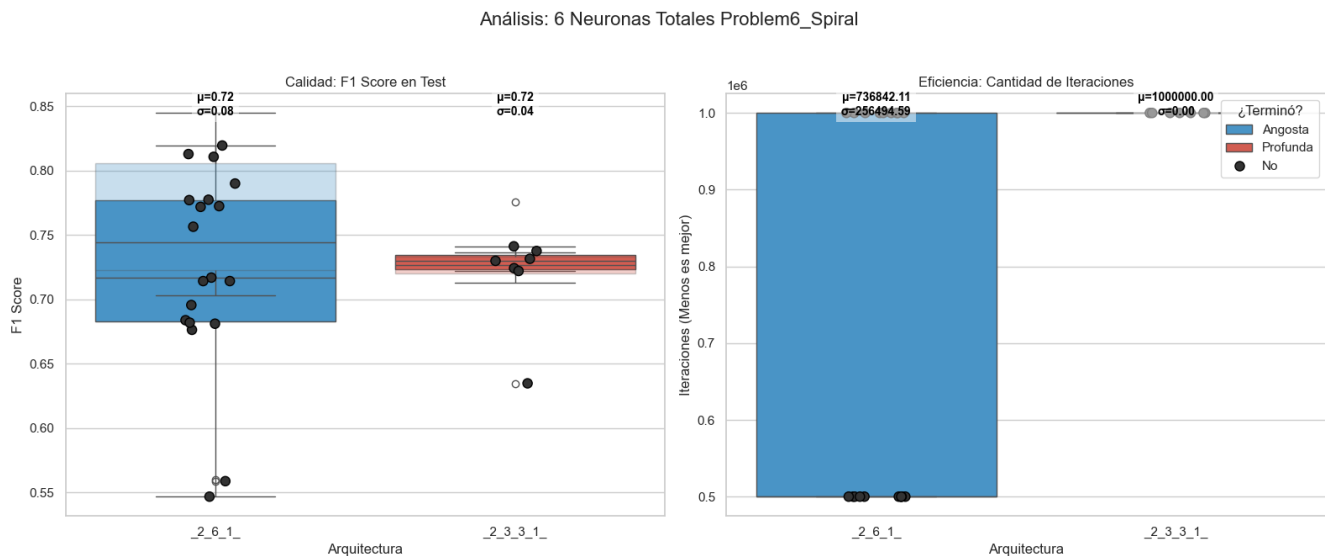


Figure 6: Performance comparison on the Spiral problem with 6 neurons.

With 9 neurons, the shallow network improves its score to 85% with a standard deviation reduced to just 2 points. In contrast, the deep network maintains a mean of 74 points but with significantly higher variance (nearly 9 points). Although there are isolated cases where the deep network manages to solve the problem, the shallow network does so in all cases and with fewer iterations.

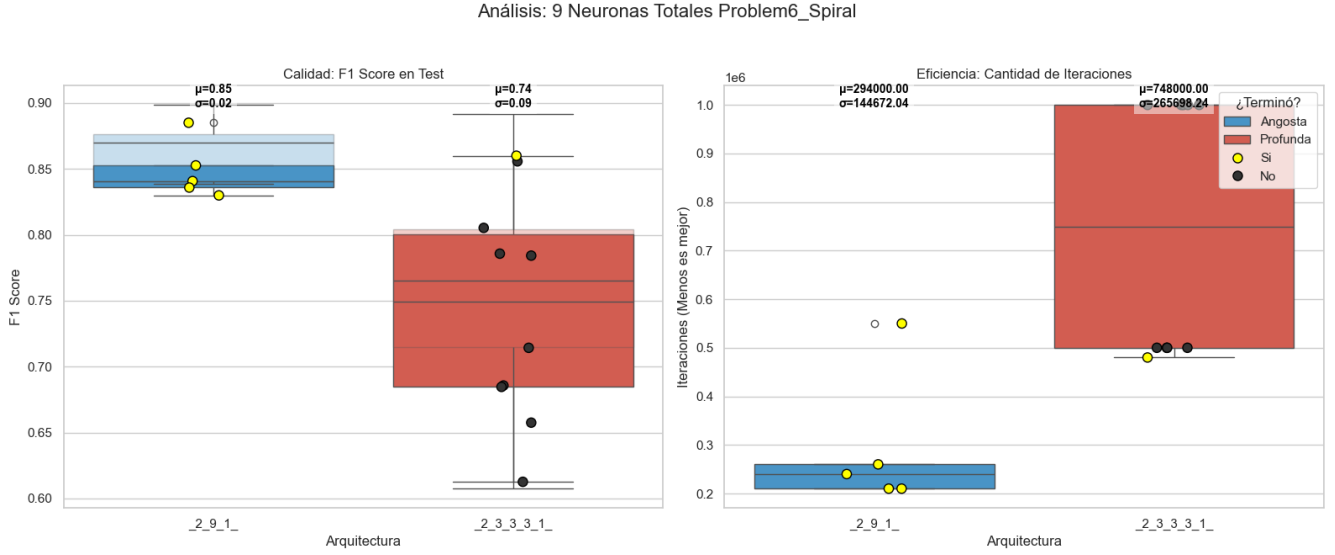


Figure 7: Performance comparison on the Spiral problem with 9 neurons.

With 12 neurons, the superiority of the shallow network is decisive, reaching an 89% score with low deviation. The deep network marginally improves its mean (2 points) but at the cost of severe instability (standard deviation of 12 points); this means that while in some cases it solves the problem in times comparable to the shallow network (200k epochs), in many others it diverges toward poor results close to 60%.

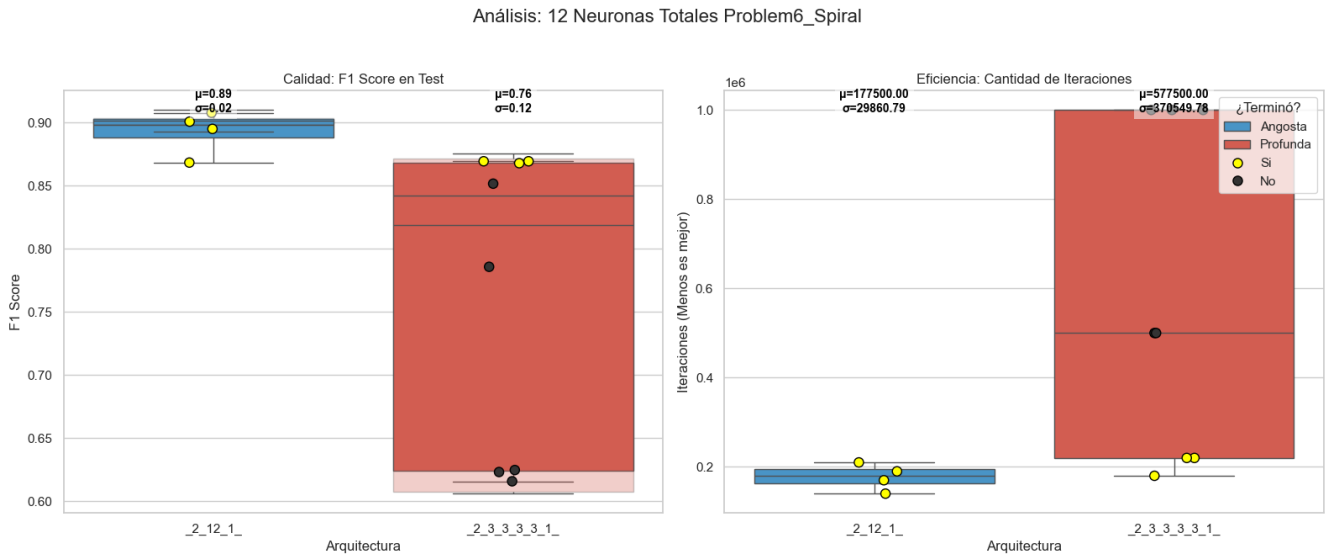


Figure 8: Performance comparison on the Spiral problem with 12 neurons.

We conclude that for this problem the shallow architecture is superior in consistency and performance, although the deep network possesses the theoretical capacity to solve it under ideal initialization conditions.

10.7 Problem 7: Sonar (Mines vs. Rocks)

- **Type:** Real dataset (Signals/Physics).
- **Description:** Sonar data bounced off a metal cylinder or a rock.
- **Challenge:** 60 frequency features and few examples (~ 208). The main challenge is the curse of dimensionality and the extreme risk of overfitting.

Results: Models of up to 6 layers (366 equivalent neurons) were tested without satisfactorily solving the problem, likely due to the scarcity of data relative to such high dimensionality.

10.8 Problem 8: Nested Circles

- **Description:** A small circle of one class surrounded by a larger ring of the other class.
- **Challenge:** Not linearly separable; requires closed curved boundaries.

Results: Once again, the shallow network shows better results, with a higher mean score and lower variance than its deep counterpart. A consistent improvement is observed, going from 89% to 95% upon reaching 12 neurons. The deep network also shows progress, albeit more slowly.

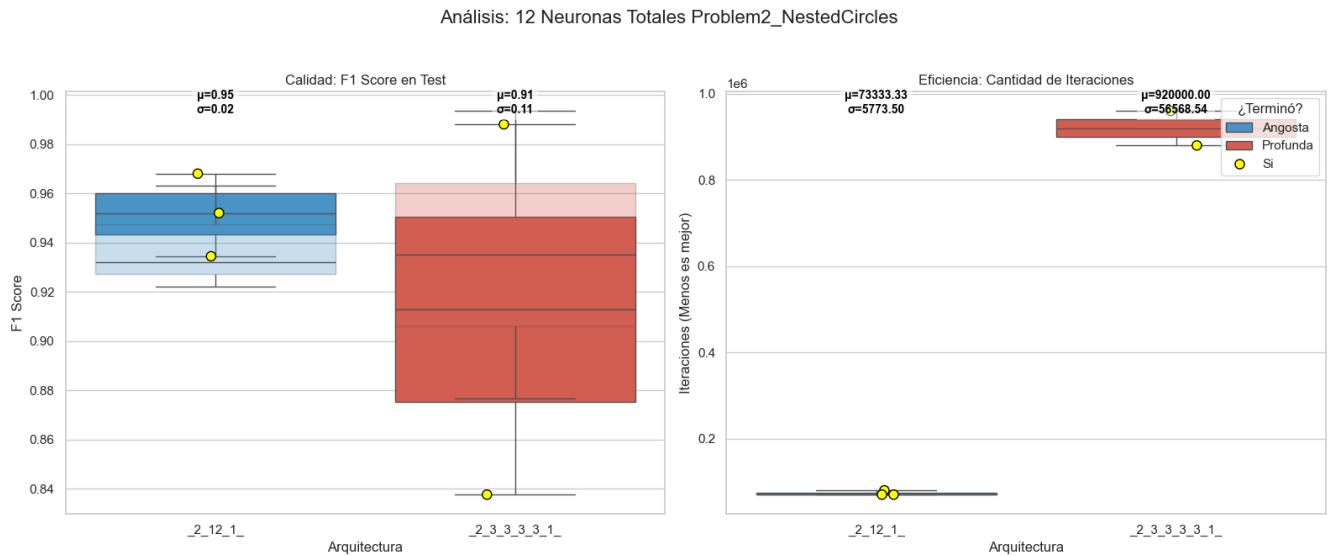


Figure 9: Comparison on Nested Circles with 12 neurons.

10.9 Problem 9: Sine Clusters

- **Description:** Classes follow interleaved wave patterns.
- **Challenge:** Wavy and periodic decision boundary.

Results: With 6 neurons, neither architecture manages to solve the problem, getting stuck at a score of around 50% (chance level).

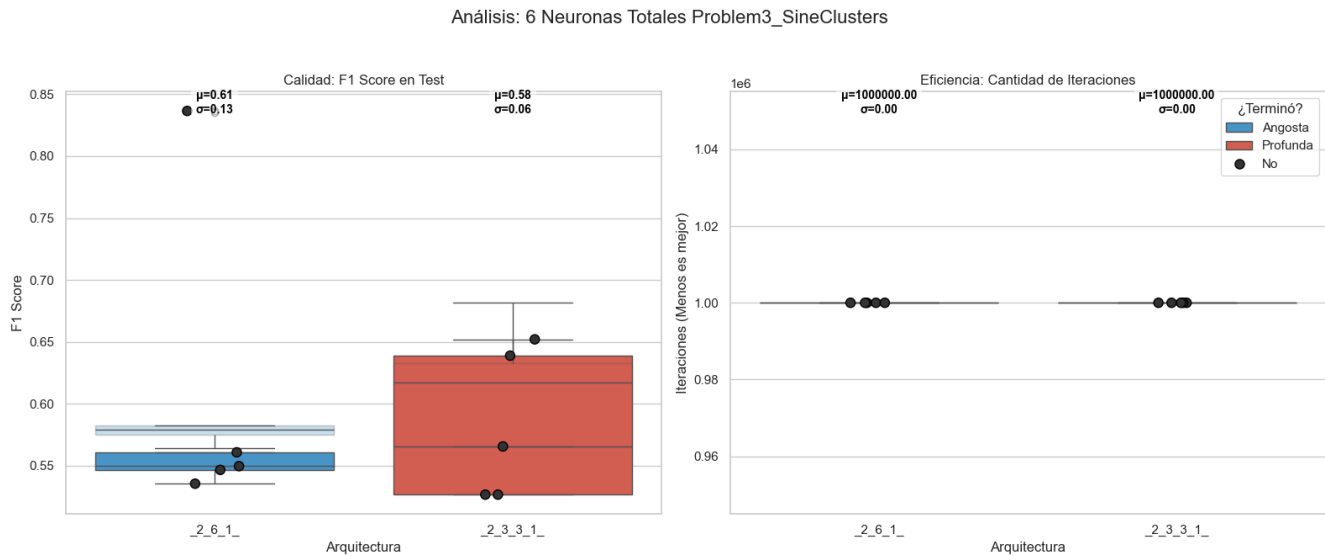


Figure 10: Performance on Sine Clusters with 6 neurons.

Upon increasing to 9 neurons, the shallow network shows a huge improvement, consistently reaching a score of 85, while the deep network lags behind at 65. Neither resolved the problem completely within the 1-million epoch limit.

With 12 neurons, the shallow network is already capable of solving the problem with a consistent 85% and minimal deviation. The deep network, for its part, fails to break past the 60% barrier.

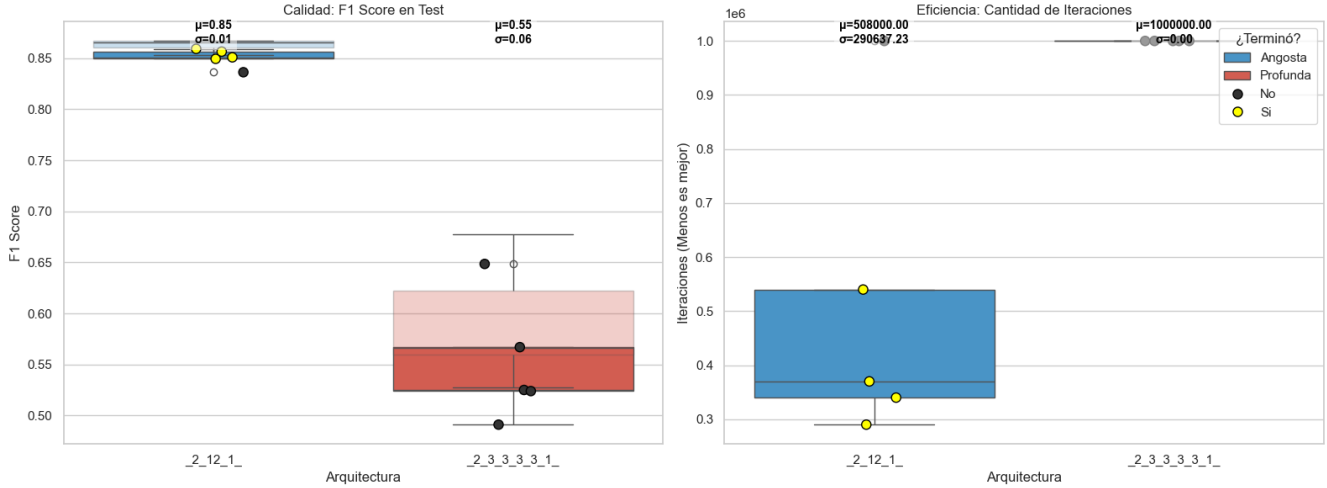


Figure 11: Performance on Sine Clusters with 12 neurons.

10.10 Problem 10: Adaptive Mesh

- **Description:** Point distribution with variable density.
- **Challenge:** Adapting the decision boundary to zones of different density.

Results: The shallow network solves this problem in record time and consistently, improving with the addition of neurons while maintaining minimal deviation (< 1). In contrast, the deep network faces growing difficulties when adding complexity; it appears to get trapped in initialization-dependent local minima. Although theoretically capable, the probability of successful convergence decreases with depth.

Análisis: 12 Neuronas Totales Problem4_AdaptiveMesh

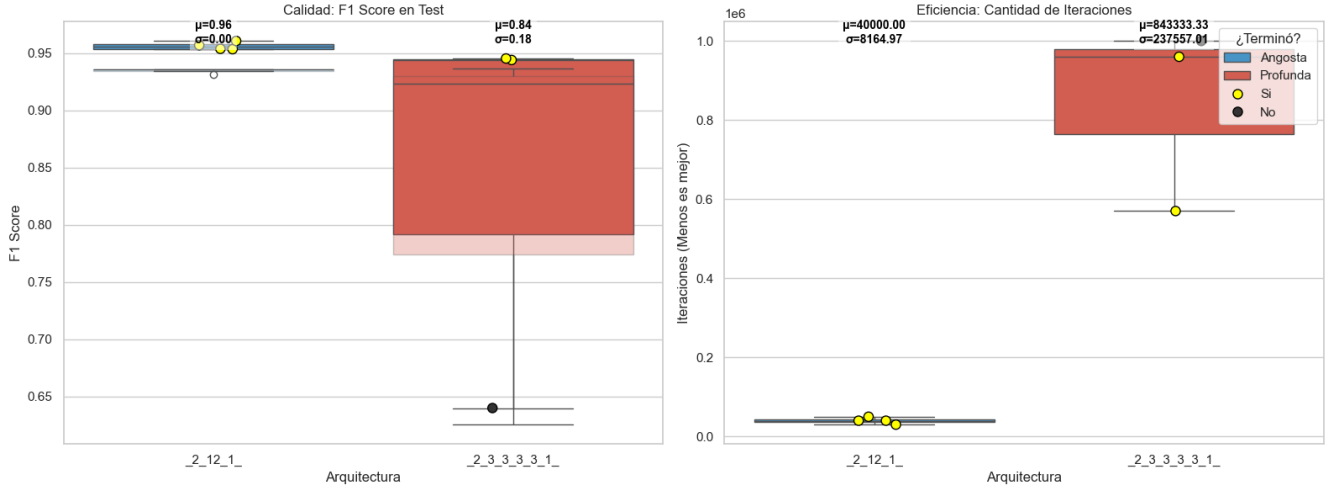


Figure 12: Performance on Adaptive Mesh.

10.11 Problem 11: Checkerboard

- **Description:** Data distributed in an alternating grid pattern.
- **Challenge:** Generalization of XOR. Requires creating disconnected decision regions.

Results: This is the first case where clear overfitting is observed, with a gap of at least 5 points between training and test scores for both architectures. With 6 and 9 neurons, neither network solves the problem satisfactorily.

Análisis: 9 Neuronas Totales Problem5_Checkerboard

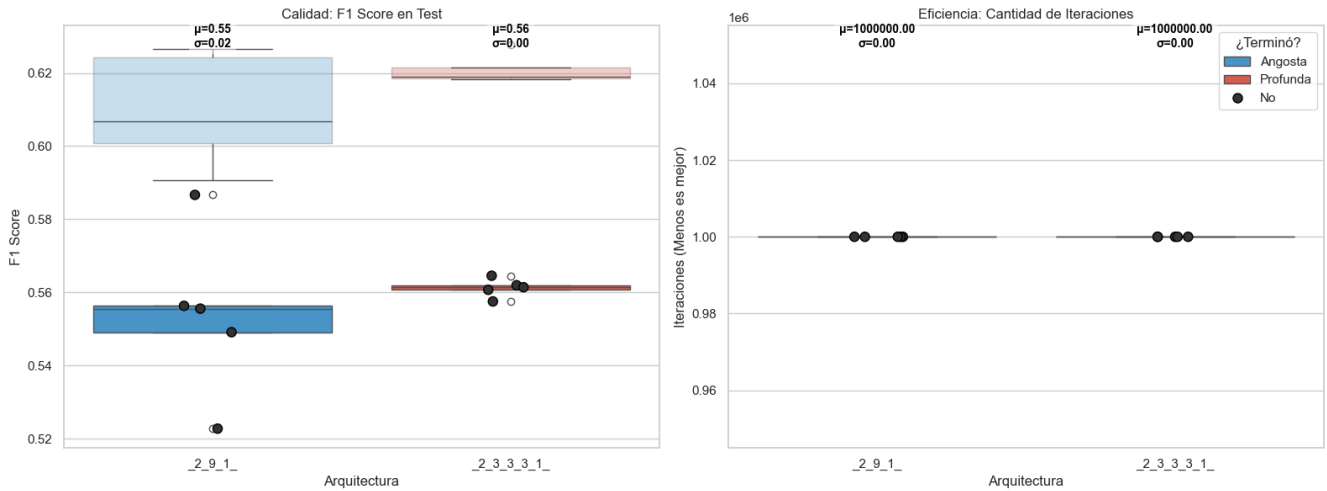


Figure 13: Performance on Checkerboard with 9 neurons.

However, upon reaching 12 neurons, an interesting phenomenon occurs: the deep network improves its performance to 64 points and significantly reduces signs of overfitting.

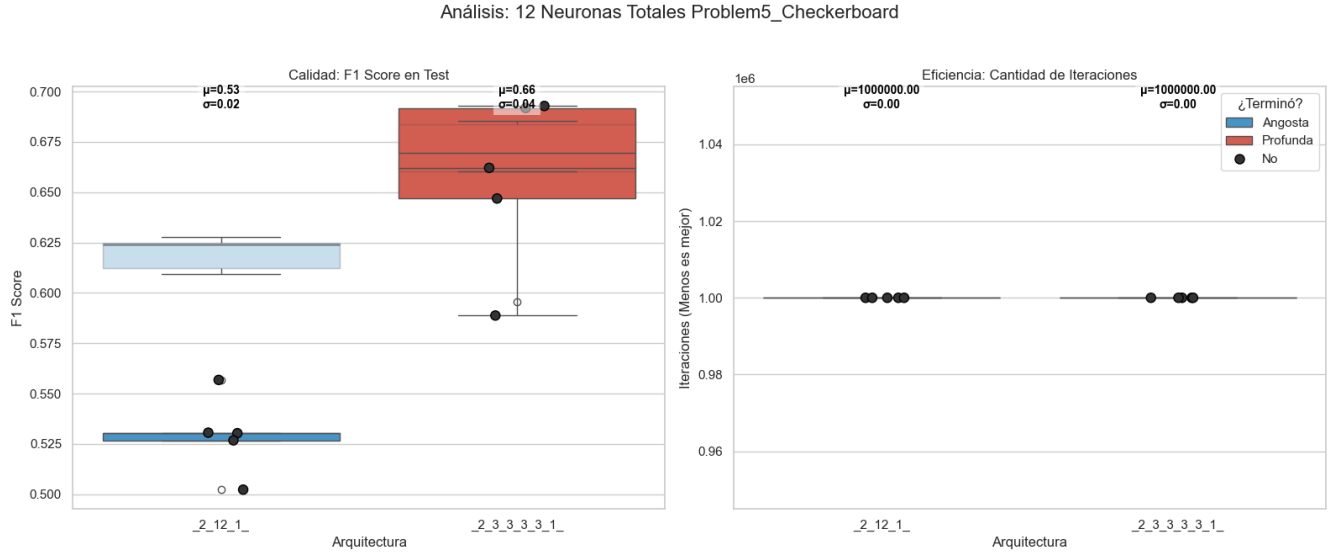


Figure 14: Performance on Checkerboard with 12 neurons.

Finally, upon adding a fourth layer (totaling 15 equivalent neurons), the deep network reaches 69 points with no appreciable variance or overfitting. The shallow network, in contrast, remains stagnant. Although the problem was not fully resolved, the trend suggests that the deep architecture could overcome it with greater depth, making this the only case where it showed clear superiority.

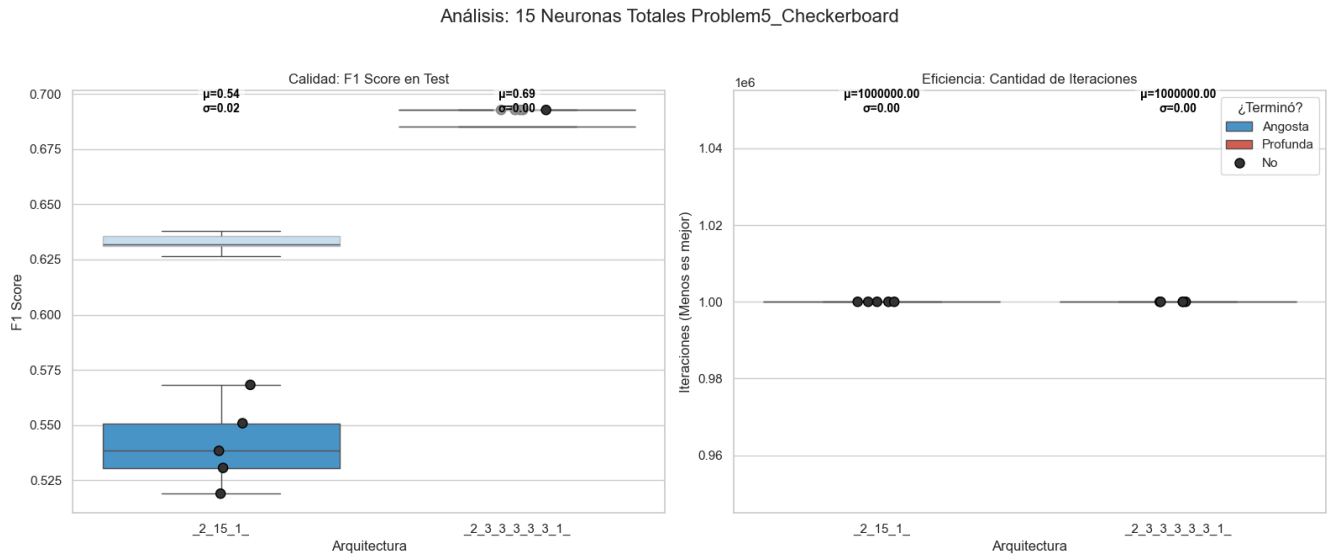


Figure 15: Performance with extended architecture (15 neurons).

10.12 Problem 12: Swiss Roll 2D

- **Description:** Dense spiral-like structure.

- **Challenge:** Manifold learning. Requires understanding structural continuity.

Results: Neither architecture solved the problem with few neurons. The deep architecture worsens its performance as layers are added, while the shallow network improves consistently but suffers from strong overfitting, ultimately stagnating at 12 neurons.

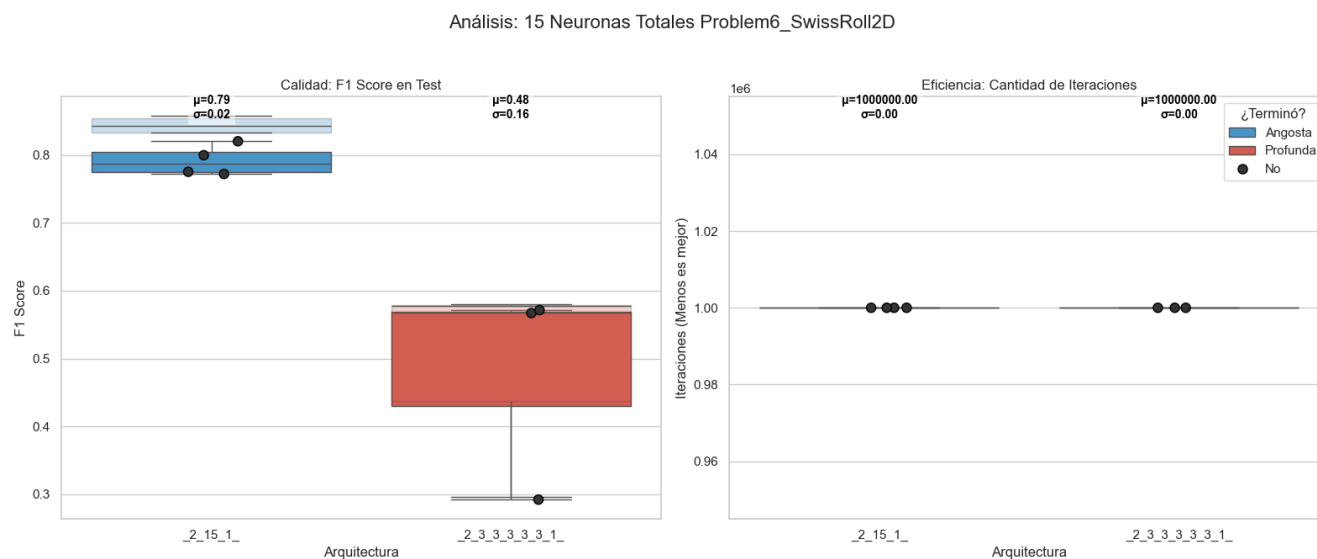


Figure 16: Results on Swiss Roll 2D.

10.13 Problem 13: Concentric Polygons

- **Description:** Nested geometric shapes with straight edges.
- **Challenge:** Tracing sharp angles and precise linear boundaries.

Results: This experiment proved unsolvable with the current procedure, obtaining very low scores of 18 and 13 points for the shallow and deep networks respectively, indicating a fundamental failure in capturing the geometry of the problem.

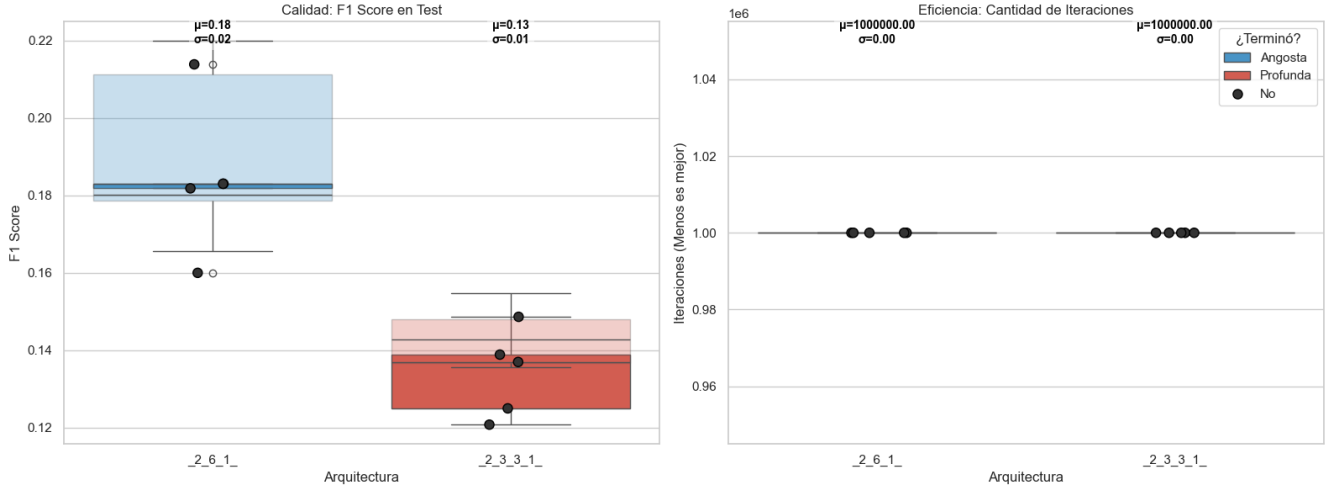


Figure 17: Poor performance on Concentric Polygons.

11 General Conclusions

After analyzing the experimental results, it appears to us that for the majority of medium-difficulty classification problems we tested, both approaches work, though with important nuances. In general, the **shallow** architecture tended to hold a slight advantage, showing faster and — most importantly — more stable convergence; when we added neurons, we almost always saw improvement, which was not always the case with deep networks.

In the cases we managed to solve, shallow networks typically yielded better scores. Nevertheless, we cannot dismiss deep networks, as they also achieved competitive solutions in several scenarios, although we noticed that they often struggled more to stabilize during training.

Something that proved very interesting to observe was the improvement pattern: in shallow networks, progress is fairly predictable as width is added to the layer. This makes us wonder whether, by analyzing these initial improvements, it would be possible to know in advance in which direction it is more advantageous to grow the network (wider or deeper) without having to blindly test everything.

The exception to this trend was the *Checkerboard* problem. Here we saw that the shallow network stagnated, while the deep architecture managed to reduce overfitting and improve performance by adding layers. This seems to indicate that, in the end, it is the nature of the problem itself that dictates the appropriate tool. After all, that is precisely why we have graphics cards and monolithic processors — each suited to its own purpose.

12 Bibliography

- Ba, Jimmy and Rich Caruana (2014). “Do Deep Nets Really Need to be Deep?” In: *Advances in Neural Information Processing Systems*.
- Chatterji, Niladri S. and Philip M. Long (2023). “Deep Linear Networks can Benignly Overfit when Shallow Ones Do Not”. In: *Journal of Machine Learning Research* 24.188, pp. 1–26.
- Cybenko, G. (1989). “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals, and Systems* 2.4, pp. 303–314. DOI: 10.1007/BF02551274.
- Fan, Simin (2024). “Deep Grokking: Would Deep Neural Networks Generalize Better?” In: arXiv: 2405.19454.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press.
- Kůrková, Věra (2020). “Limitations of Shallow Networks”. In: *Recent Trends in Learning From Data*. Springer. DOI: 10.1007/978-3-030-43883-8_6.
- Lu, Zhou et al. (2017). “The Expressive Power of Neural Networks: A View from the Width”. In: *Advances in Neural Information Processing Systems*.
- Mhaskar, Hrushikesh, Qianli Liao, and Tomaso Poggio (2017). “When and Why Are Deep Networks Better Than Shallow Ones?” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. DOI: 10.1609/aaai.v31i1.10913.
- Nielsen, Michael A. (2015). *Neural Networks and Deep Learning*.
- Radhakrishnan, Adityanarayanan, Mikhail Belkin, and Caroline Uhler (2023). “Wide and deep neural networks achieve consistency for classification”. In: *Proceedings of the National Academy of Sciences* 120.15, e2208779120. DOI: 10.1073/pnas.2208779120.
- Yang, Yahong and Juncai He (2024). “Deeper or Wider: A Perspective from Optimal Generalization Error with Sobolev Loss”. In: arXiv: 2402.00152.